Actividad 3. Taller propuesto
Reconocimiento de caracteres manuscritos japoneses con una red convolucional

Informe de ejecución

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002

Fecha: 10 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Índice

1. Introducción **2**

2. Marco teórico (resumen) **2**
2.1. Convolución, pooling y la capa final . . . 2
2.2. Normalización por lotes y dropout . . . 2
2.3. Aumento de datos . . . 2

3. Metodología **3**
3.1. Conjunto de datos . . . 3
3.2. Arquitectura propuesta . . . 3
3.3. Entrenamiento, evaluación y aumento . . . 3

4. Resultados **4**
4.1. Exploración del conjunto . . . 4
4.2. Entrenamiento . . . 5
4.3. Evaluación en prueba . . . 5
4.4. Distancia al entrenamiento . . . 6
4.5. Aumento de datos . . . 7

5. Análisis **8**
5.1. Por qué cada componente de la red . . . 8
5.2. La brecha entre validación y prueba . . . 9
5.3. Por qué se confunden su y tsu . . . 9
5.4. Qué hizo y qué no hizo el aumento de datos . . . 9

6. Conclusiones **10**

7. Referencias **10**

A. Código de los bloques completados **10**

1. Introducción

El cuaderno resuelto del docente clasifica los dígitos de MNIST con una red convolucional ya diseñada. El cuaderno propuesto pide repetir el flujo con otro conjunto de datos y completar cinco bloques `TODO`. Esos bloques exploran el conjunto, definen una red propia con ciertas restricciones, escriben el bucle de entrenamiento, evalúan en prueba y explican el error más frecuente, e implementan un aumento de datos y miden su efecto. Se eligió KMNIST (Clanuwat et al., 2018), caracteres manuscritos japoneses en escritura cursiva antigua, con el mismo formato que MNIST, imágenes de 28×28 en escala de grises y diez clases, pero con formas mucho más variables dentro de cada clase.

El cuaderno se ejecutó completo con PyTorch 2.12 (Paszke et al., 2019) en GPU, semilla 42 y 5 épocas. Se descargó el conjunto real, sin el respaldo que el cuaderno usa cuando falla la descarga, y las once verificaciones automáticas dieron `[OK]`. La celda opcional que exporta las imágenes a carpetas para MATLAB no se ejecutó porque el trabajo se hizo en Python. Se agregaron tres celdas de análisis, una que muestra imágenes del par de clases más confundido, una que mide la distancia de cada imagen de validación y de prueba a su vecino más cercano en el entrenamiento, y una que repite la comparación del aumento de datos con tres semillas, como recomienda el aviso del propio cuaderno. El código de los cinco bloques está en el Anexo A.

2. Marco teórico (resumen)

2.1 Convolución, pooling y la capa final

Una capa convolucional desliza filtros de 3×3 sobre la imagen y produce un mapa de activación por filtro. Con relleno de un píxel el mapa conserva el tamaño de la entrada, 28×28, y es el *max pooling* de 2×2 el que reduce a 14 y luego a 7 píxeles por lado, quedándose con el valor más alto de cada ventana. Después de dos bloques quedan 64 mapas de 7×7, 3 136 características, que una capa lineal convierte en diez *logits*. La softmax y la entropía cruzada van dentro de la pérdida, y el optimizador es Adam (Kingma y Ba, 2015), como en el cuaderno resuelto.

2.2 Normalización por lotes y dropout

La normalización por lotes (Ioffe y Szegedy, 2015) reescala las activaciones de cada canal para que en cada lote tengan media cero y varianza uno, y después aplica una escala y un desplazamiento que la red aprende, dos parámetros por canal. En evaluación no usa el lote sino promedios móviles de la media y la varianza acumulados durante el entrenamiento, y por eso importa pasar la red a modo de evaluación antes de medirla. Sus autores la explicaron como un remedio al cambio de escala de las activaciones entre capas, aunque trabajos posteriores atribuyen su efecto a que suaviza la superficie de la pérdida (Santurkar et al., 2018). En la práctica permite entrenar con tasas de aprendizaje mayores y converger en menos épocas. El *dropout* (Srivastava et al., 2014) apaga al azar una fracción p de las características durante el entrenamiento y reescala las restantes por $1/(1-p)$, de modo que la capa siguiente no puede depender de unas pocas, y se desactiva en la evaluación.

2.3 Aumento de datos

El aumento de datos crea variantes de las imágenes de entrenamiento, aquí desplazamientos de hasta dos píxeles en cada eje, para que la red vea en cada época una versión distinta de cada imagen. Actúa como regularizador, hace el entrenamiento más difícil y, si el conjunto de prueba contiene las mismas variaciones, mejora la generalización. La forma de comprobarlo es comparar con y sin

aumento con la misma semilla y el mismo número de épocas, y repetir con varias semillas antes de concluir.

3. Metodología

3.1 Conjunto de datos

KMNIST tiene 60 000 imágenes de entrenamiento y 10 000 de prueba, con una clase por cada uno de diez caracteres del silabario *hiragana* en su escritura antigua, *o*, *ki*, *su*, *tsu*, *na*, *ha*, *ma*, *ya*, *re* y *wo*. Cada clase agrupa varias formas de escribir el mismo sonido, porque antes de la reforma de 1900 un sonido podía escribirse con caracteres de distinto origen, y eso se ve en la Figura 1. Según sus autores las imágenes de entrenamiento y de prueba salen de los mismos 35 libros impresos. El cuaderno aparta 5 000 imágenes como validación, entrena con 55 000 y normaliza con la media (0.1918) y la desviación (0.3484) del entrenamiento.

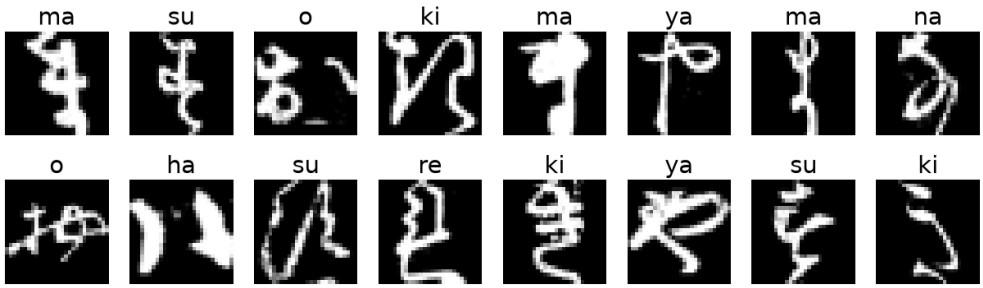


Figura 1: Primeras dieciséis imágenes de entrenamiento de KMNIST con su clase

3.2 Arquitectura propuesta

El enunciado pide al menos dos bloques de convolución con normalización por lotes y *pooling*, *dropout* antes de la capa final y menos de 200 000 parámetros. La red de la Tabla 1 y la Figura 2 cumple con 50 378 parámetros, una cuarta parte del límite. Cada componente tiene una razón. La convolución con relleno conserva el tamaño para que sea el *pooling* quien decida la reducción. Los 32 y 64 filtros duplican los 16 y 32 de la red del cuaderno resuelto porque las formas de KMNIST son más variadas que las de los dígitos. La normalización por lotes va justo después de cada convolución y antes de la ReLU, el orden habitual, con la expectativa de un arranque rápido con la tasa de 10^{-3}. El *dropout* de 0.3 protege la capa lineal, que concentra el 62 % de los parámetros y es donde más riesgo hay de memorizar. El valor 0.3 es un punto medio entre el 0.25 del cuaderno resuelto y el 0.5 usual en capas densas, y no se ajustó.

3.3 Entrenamiento, evaluación y aumento

El bucle de entrenamiento sigue los cinco pasos de cada iteración, borrar los gradientes, propagar el lote, calcular la entropía cruzada, retropropagar y actualizar con Adam a tasa 10^{-3}, con lotes de 64 imágenes y 5 épocas. Como en el cuaderno resuelto, la pérdida y la exactitud de entrenamiento son el promedio de los lotes de la época, medido mientras los pesos cambian y con el *dropout* activo, y las de validación se miden al final de la época en modo de evaluación.

La prueba se evaluó una vez con la red principal y ocho veces más dentro de la comparación del aumento, dos en la celda del cuaderno y seis en la repetición con tres semillas. Ninguna decisión

Tabla 1: Capas de la red propuesta. Las convoluciones usan relleno de un píxel y conservan el tamaño

Bloque	Capa	Parámetros
1	Convolución 3×3, 32 filtros, salida $32 \times 28 \times 28$	320
	Normalización por lotes, ReLU, max pooling 2×2, salida $32 \times 14 \times 14$	64
2	Convolución 3×3, 64 filtros, salida $64 \times 14 \times 14$	18 496
	Normalización por lotes, ReLU, max pooling 2×2, salida $64 \times 7 \times 7$	128
Salida	Aplanar, 3 136, y dropout 0.3	0
	Lineal 3 136 a 10	31 370
	Total	50 378

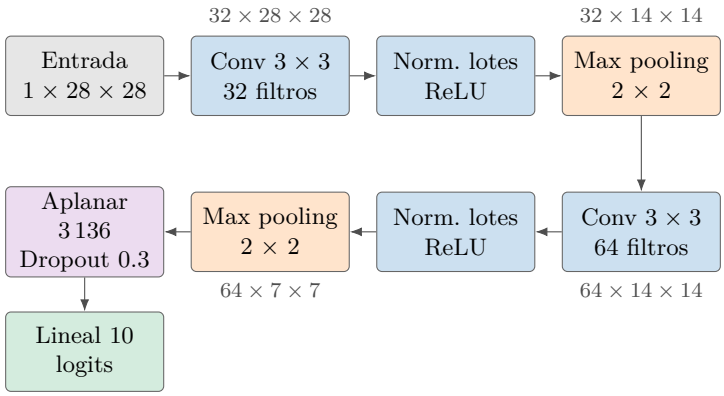


Figura 2: Red propuesta, 50 378 parámetros. Junto a cada bloque va el tamaño de su salida

de diseño se tomó mirando esas cifras. El aumento desplaza cada lote completo un número entero de píxeles entre -2 y 2 en cada eje con `torch.roll`, de modo que las 64 imágenes del lote reciben el mismo desplazamiento y lo que sale por un borde entra por el opuesto. La comparación con y sin aumento se repite con las semillas 42, 43 y 44. La semilla fija la inicialización de los pesos y los desplazamientos, pero no el orden de los lotes, que depende de un generador propio del cargador de datos que el cuaderno no reinicia. Por eso dos corridas con la misma semilla no son idénticas y la comparación no está emparejada al detalle, lo que obliga a leer las diferencias frente a la dispersión entre semillas.

Para la distancia al entrenamiento se aplanó cada imagen normalizada a un vector de 784 valores y se calculó la distancia euclidiana a la imagen de entrenamiento más cercana, para las 5 000 de validación y las 10 000 de prueba.

4. Resultados

4.1 Exploración del conjunto

Las diez clases tienen entre 5 450 y 5 533 imágenes de entrenamiento, una diferencia menor al 2 %, y la prueba tiene exactamente 1 000 por clase. El conjunto está balanceado, la exactitud global es una medida fiable y no hace falta ponderar las clases ni remuestrear.

4.2 Entrenamiento

La Tabla 2 y la Figura 3 muestran las 5 épocas. La validación llega a 95.84 % en la primera época y sube hasta 97.66 en la época 4, donde su pérdida también es mínima. En la última época la validación retrocede a 97.34 % y su pérdida sube, mientras que el entrenamiento sigue mejorando hasta 97.53 %. Una parada temprana sobre la validación habría conservado la red de la época 4.

Tabla 2: Pérdida y exactitud por época de la red propuesta. La exactitud está en porcentaje. Las columnas de entrenamiento son promedios de la época y las de validación se miden al final de ella

	Entrenamiento		Validación	
Época	**Pérdida**	**Exactitud**	**Pérdida**	**Exactitud**
1	0.3056	90.71	0.1392	95.84
2	0.1534	95.34	0.1032	96.62
3	0.1175	96.42	0.0908	97.40
4	0.0944	97.08	0.0809	97.66
5	0.0813	97.53	0.0893	97.34

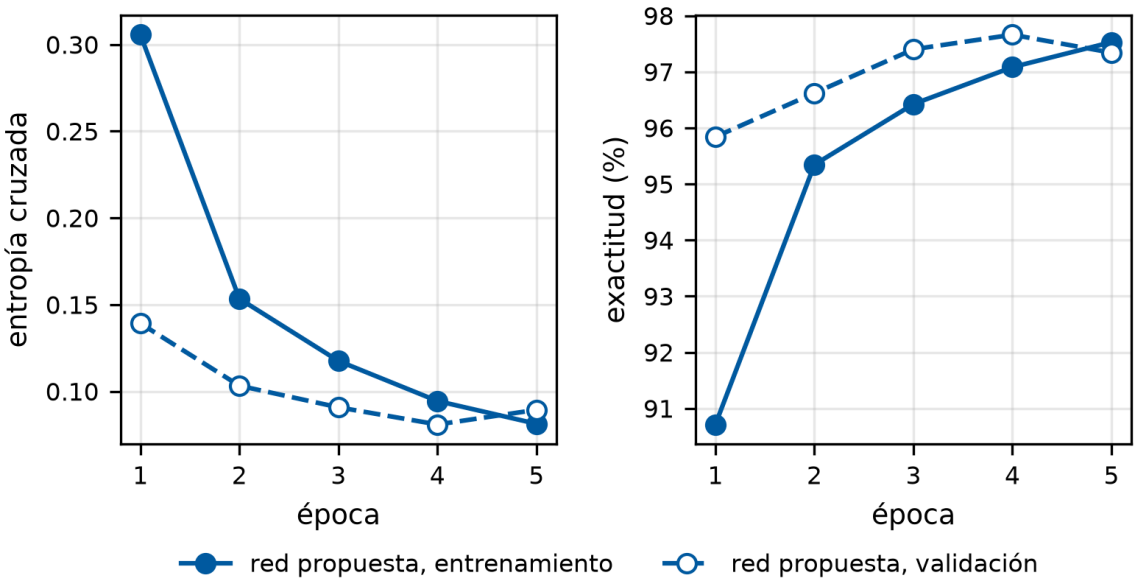


Figura 3: Pérdida y exactitud por época de la red propuesta en KMNIST. Línea continua y marcador lleno para el entrenamiento, discontinua y marcador hueco para la validación

4.3 Evaluación en prueba

La exactitud en prueba es 92.88 %, 712 errores de 10 000, con pérdida 0.2792, 4.5 puntos por debajo de la validación. La Figura 4 muestra la matriz de confusión y la Tabla 3 el acierto por clase, que va de 82.9 % en su y 84.8 en ya a 96.3 en na. El par más confundido es su tomado por tsu, 52 de las 1 000 imágenes de su. Ese par explica menos de un tercio de los 171 errores de su, el resto va sobre todo a na (33), ha (33) y ma (30). La columna na recibe 229 errores de las nueve clases restantes, el 32 % del total, y por eso na tiene la precisión más baja, 80.8 %, aunque su acierto sea el más alto. La Figura 5 muestra las cinco primeras imágenes de su tomadas por tsu junto a imágenes bien clasificadas de las dos clases.

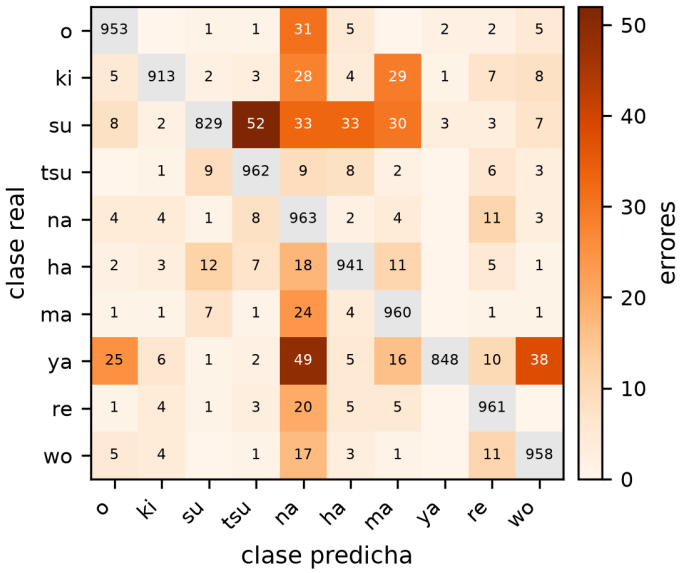


Figura 4: Matriz de confusión sobre las 10 000 imágenes de prueba. El color codifica solo los errores y la diagonal va en gris para que los aciertos no aplanen la escala

Tabla 3: Acierto por clase en prueba. El acierto es la fracción de imágenes de la clase bien clasificadas y la precisión la fracción de predicciones de esa clase que eran correctas

Clase	Imágenes	Aciertos	Acierto (%)	Precisión (%)
o	1000	953	95.3	94.9
ki	1000	913	91.3	97.3
su	1000	829	82.9	96.1
tsu	1000	962	96.2	92.5
na	1000	963	96.3	80.8
ha	1000	941	94.1	93.2
ma	1000	960	96.0	90.7
ya	1000	848	84.8	99.3
re	1000	961	96.1	94.5
wo	1000	958	95.8	93.6

4.4 Distancia al entrenamiento

La Tabla 4 compara la distancia de cada imagen de validación y de prueba a su vecino más cercano en el entrenamiento. Ninguna imagen de validación ni de prueba coincide con una de entrenamiento. La prueba queda algo más lejos que la validación en la mediana y en los dos percentiles, 3.6 % más en la mediana.

Tabla 4: Distancia euclidiana al vecino más cercano del entrenamiento, sobre las imágenes normalizadas

Conjunto	Percentil 10	Mediana	Percentil 90
Validación	13.6	18.9	24.4
Prueba	14.2	19.6	25.9

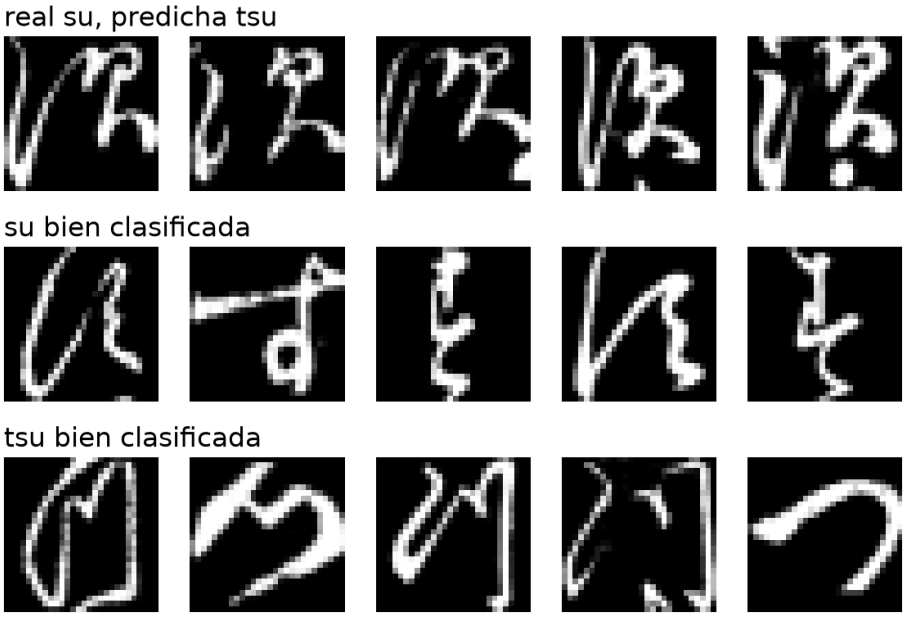


Figura 5: Arriba, las cinco primeras imágenes de su que la red tomó por tsu. En medio y abajo, las cinco primeras imágenes de su y de tsu bien clasificadas

4.5 Aumento de datos

La Figura 6 muestra el efecto del desplazamiento sobre cuatro imágenes. La Tabla 5 y la Figura 7 resumen las tres semillas con 5 épocas cada una, y la Tabla 6 da la diferencia por semilla. Con aumento la exactitud de entrenamiento baja 2.3 puntos en las tres semillas. En validación y en prueba las diferencias medias son de 0.0 y -0.2 puntos, menores que la desviación entre semillas, y por semilla ni siquiera coinciden en el signo. La brecha entre validación y entrenamiento pasa de 0.2 a 2.6 puntos.

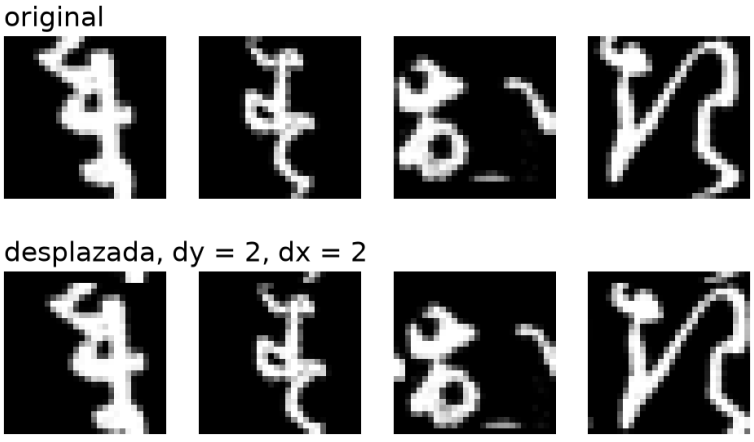


Figura 6: Cuatro imágenes de entrenamiento antes y después del desplazamiento, con el caso extremo del rango, 2 píxeles hacia abajo y 2 a la derecha. Los trazos que tocaban el borde inferior reaparecen arriba

Tabla 5: Exactitud en porcentaje, media y desviación muestral de las semillas 42, 43 y 44 con 5 épocas

	Entrenamiento	**Validación**	**Prueba**
Sin aumento	97.49 ± 0.04	97.71 ± 0.13	93.10 ± 0.32
Con aumento	95.17 ± 0.18	97.73 ± 0.33	92.88 ± 0.44

Tabla 6: Exactitud de validación y de prueba por semilla, en porcentaje, y diferencia con aumento menos sin aumento

	Validación		**Prueba**		**Diferencia**	
Semilla	**Sin**	**Con**	**Sin**	**Con**	**Validación**	**Prueba**
42	97.84	97.40	93.24	92.69	−0.44	−0.55
43	97.58	98.06	92.73	93.38	+0.48	+0.65
44	97.70	97.72	93.32	92.57	+0.02	−0.75

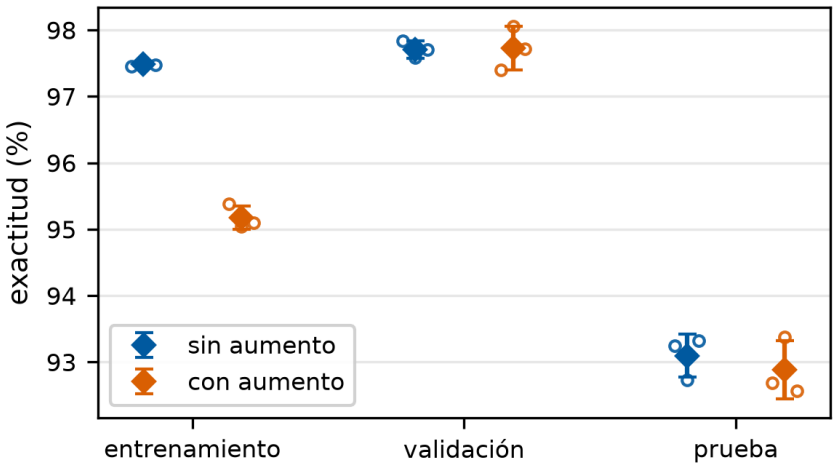


Figura 7: Exactitud con y sin aumento. Los círculos huecos son las tres semillas y el rombo la media con la desviación muestral

5. Análisis

5.1 Por qué cada componente de la red

La pregunta del paso 2 era qué se esperaba de cada componente. La convolución aporta filtros que se reutilizan en toda la imagen, y con 32 y 64 de ellos la red tiene margen para las variantes de forma de KMNIST. La normalización por lotes se incluyó esperando un arranque rápido, y la red llegó a 95.84 % de validación en la primera época, pero sin una corrida sin normalización no se puede separar su aporte del resto. El *pooling* redujo las 50 176 activaciones del segundo bloque a las 3 136 que entran a la capa lineal, y aun así esa capa concentra el 62 % de los parámetros, lo que justifica el *dropout* delante de ella. La Tabla 2 muestra que la combinación funcionó, la pérdida de validación baja hasta la época 4 y la exactitud de entrenamiento alcanza a la de validación en la última. Que la validación empeore en la época 5 mientras el entrenamiento mejora es el primer signo de que, con más épocas, el *dropout* de 0.3 no bastaría.

5.2 La brecha entre validación y prueba

La validación termina en 97.34 % y la prueba en 92.88, 4.5 puntos menos. En el cuaderno resuelto la prueba de MNIST quedó una centésima por encima de la validación. No es un sobreajuste a la validación, que nunca se usó para elegir nada, y tampoco se explica por la procedencia de los datos, porque según sus autores las imágenes de entrenamiento y de prueba de KMNIST salen de los mismos 35 libros. Una hipótesis compatible con los datos es que la validación, tomada al azar del entrenamiento, comparte páginas y manos con él y resulta optimista, mientras que la prueba contiene formas menos representadas. La Tabla 4 apunta en esa dirección, la prueba queda sistemáticamente más lejos del entrenamiento que la validación, pero la diferencia es pequeña y no alcanza para atribuirle los 4.5 puntos. Lo que sí se puede afirmar es que la validación interna no anticipa el resultado en prueba de este conjunto, y que la brecha es conocida. En el artículo de KMNIST una red convolucional sencilla de dos convoluciones entrenada 12 épocas alcanza 95.12 % de prueba, un clasificador de vecinos más cercanos 91.56 y una red residual 97.82. La red de este taller, con 5 épocas, queda por debajo de la convolucional sencilla, lo que indica que la parada en 5 épocas también cuesta puntos.

5.3 Por qué se confunden su y tsu

La clase su tiene el acierto más bajo. Reúne varias formas antiguas del carácter, y de sus 171 errores 52 van a tsu y el resto se reparte entre na, ha y ma. La Figura 5 muestra por qué con tsu. Las cinco imágenes mal clasificadas son la misma variante, un trazo vertical largo a la izquierda y un grupo de trazos a la derecha, y se parecen a la variante de tsu de trazos verticales que aparece en la fila inferior. Entre las su bien clasificadas también hay formas con el trazo vertical largo, así que la red distingue esa variante en algunos casos y no en otros. Cinco imágenes elegidas por índice no bastan para generalizar, pero una explicación compatible con la figura es que una misma clase contiene variantes que se parecen más a otra clase que entre sí. La columna na con el 32 % de los errores muestra otra cosa, na recibe imágenes de las nueve clases restantes, lo que sugiere que sus trazos curvos y sueltos comparten fragmentos con muchos caracteres. La clase ya, segunda con menos acierto, también tiene a na como destino más frecuente de sus errores, como se ve en la Figura 4.

5.4 Qué hizo y qué no hizo el aumento de datos

El aumento no mejoró la exactitud de validación ni la de prueba. Las diferencias medias son menores que la dispersión entre semillas y cambian de signo de una semilla a otra, así que no se puede afirmar un cambio en ningún sentido. Lo que sí cambió es la brecha, que pasó de 0.2 a 2.6 puntos porque el entrenamiento cayó 2.3 puntos. Esa caída se mide sobre las imágenes desplazadas, y muestra que la red no era invariante a desplazamientos de uno o dos píxeles, porque si lo fuera las imágenes desplazadas no le costarían nada. La capa lineal sobre los 64 mapas de 7×7 depende de la posición, y el *pooling* solo da tolerancia local.

Hay tres razones para que el aumento no ayudara en prueba. La primera es de implementación. Las 64 imágenes del lote reciben el mismo desplazamiento, lo que da poca diversidad, y `torch.roll` hace entrar por el borde opuesto los trazos que tocan el marco, algo frecuente en KMNIST como se ve en la Figura 6, con lo que una parte de las imágenes aumentadas no son variantes realistas. La segunda es que en 5 épocas el sobreajuste apenas empieza, y un regularizador tiene poco que corregir. La tercera es que los errores de prueba parecen venir de la forma del carácter y no de su posición, como sugiere el par su y tsu, y un desplazamiento no cambia la forma. Si se quisiera cerrar la brecha de prueba haría falta un desplazamiento por imagen con relleno de ceros en lugar de

envolvente, aumentos que cambien la forma, como rotaciones o deformaciones pequeñas del trazo, y más épocas.

6. Conclusiones

La red propuesta de dos bloques con normalización por lotes y *dropout* alcanza 97.34 % de validación y 92.88 de prueba en KMNIST con 50 378 parámetros, y cumple todas las restricciones del enunciado con las once verificaciones en `[OK]`.

La brecha de 4.5 puntos entre validación y prueba no es sobreajuste a la validación ni una diferencia de procedencia de los datos. La prueba queda algo más lejos del entrenamiento que la validación, pero esa medida solo explica una parte, y el resultado principal es que en este conjunto la validación interna no anticipa la prueba.

El error dominante, su tomado por tsu en 52 de 1 000 imágenes, nace de una clase que agrupa variantes de escritura muy distintas, y la columna na concentra el 32 % de los errores con imágenes de todas las demás clases.

El aumento por desplazamiento de dos píxeles no produjo en validación ni en prueba un cambio distinguible de la variación entre semillas, y bajó 2.3 puntos la exactitud de entrenamiento. La red no era invariante al desplazamiento, pero el aumento tal como está implementado, con un solo desplazamiento por lote y envolvente en los bordes, no genera variantes que ayuden en prueba.

Repetir el experimento con tres semillas fue necesario. Con una sola semilla el signo de la diferencia en prueba depende de cuál se elija.

7. Referencias

- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep learning for classical Japanese literature. *NeurIPS Workshop on Machine Learning for Creativity and Design*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning*.
- Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How does batch normalization help optimization? *Advances in Neural Information Processing Systems*, 31.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- Paszke, A. et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

A. Código de los bloques completados

Se reproducen los cinco bloques `TODO` tal como quedaron en el cuaderno entregado, delimitados con marcas del mismo estilo que usa el docente.

Código 1: TODO 1, imágenes por clase

```
conteo = torch.bincount(datos["y_tr"], minlength=len(clases))
```

Código 2: TODO 2, arquitectura

```
class CNNPropia(nn.Module):
    def __init__(self, p_drop=0.3):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)     # 28 -> 28
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)    # 14 -> 14
        self.bn2 = nn.BatchNorm2d(64)
        self.drop = nn.Dropout(p_drop)
        self.fc = nn.Linear(64 * 7 * 7, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.bn1(self.conv1(x))), 2)        # 28 -> 14
        x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)        # 14 -> 7
        return self.fc(self.drop(x.flatten(1)))
```

Código 3: TODO 3, los cinco pasos de una iteración

```
opt.zero_grad()
salida = modelo(xb)
perdida = F.cross_entropy(salida, yb)
perdida.backward()
opt.step()
```

Código 4: TODO 4, par más confundido

```
fuera = cm.copy(); np.fill_diagonal(fuera, 0)
par = tuple(int(v) for v in np.unravel_index(np.argmax(fuera), fuera.shape))
```

Código 5: TODO 5, desplazamiento aleatorio del lote

```
def aumentar_desplazamiento(xb, max_px=2):
    dy, dx = (int(v) for v in torch.randint(-max_px, max_px + 1, (2,)))
    return torch.roll(xb, shifts=(dy, dx), dims=(2, 3))
```